

AICL Editor

Web-Based Development Environment

User Manual

Version 1.0.0

Philippe-Antoine | June 2026

1. Introduction

The AICL Web Editor is a browser-based development environment for the Architecture Compilation Language, inspired by Python's IDLE but designed specifically for AICL's unique workflow. Like IDLE provides an integrated editor, shell, and debugger for Python, the AICL Editor provides a specification editor, compilation engine, proof viewer, and interactive REPL for AICL. The editor features a dark theme, AICL syntax highlighting, multi-file tabs, and one-click access to all AICL compilation, verification, and audit features.

The editor is organized into four main areas: the toolbar at the top for actions, the left panel for examples and exercises, the center panel for code editing, and the right panel for output and generated code. An interactive REPL at the bottom provides a command-line interface similar to the TUI, allowing you to type commands and AICL code directly.

2. Interface Layout

2.1 Toolbar

The toolbar provides one-click access to all AICL operations, organized from left to right:

New - Create a new untitled .aicl file in a new tab

Open - Open a .aicl file from your local filesystem

Save - Download the current file as .aicl

Target selector - Choose compilation target (Python, Rust, JavaScript, Go)

Compile - Compile the current AICL source to the selected target

Verify - Check specification completeness, coherence, and satisfaction

Audit - Run audit on compiled output (coverage, orphan detection)

Explain - Show provenance explanation for each compilation decision

Tree - Display the architecture tree of the current specification

Optimize - Suggest architectural optimizations

Clear - Clear the output panel

Find - Search and replace within the editor

2.2 Left Panel: Examples and Exercises

The left panel has two tabs: Examples and Exercises. The Examples tab lists four built-in AICL programs ranging from simple (Blue Square, Level 1) to complex (Chess, Level 9). Click any example to load it into the editor. The Exercises tab lists five progressive exercises with descriptions and starter templates. Click an exercise to load its template with TODO markers.

2.3 Center Panel: Code Editor

The code editor supports AICL syntax highlighting with keywords in cyan, types in green, and comments in gray. It includes line numbers, auto-indentation (4 spaces after colons), Tab/Shift+Tab for indent/outdent, and Ctrl+F for find and replace. Multiple files can be open simultaneously in tabs at the top of the editor. Each tab shows the filename and a close button. The current file is highlighted.

2.4 Right Panel: Output and Generated Code

The right panel has three tabs: Output, Tree, and Code. The Output tab shows compilation results, verification reports, and audit summaries in a log format with timestamps. The Tree tab displays the architecture tree visualization. The Code tab shows the generated source code and test code with copy buttons for easy extraction.

2.5 Bottom Panel: REPL and Exercises

The bottom panel has three tabs: Output (duplicate of right panel output for convenience), REPL (an interactive command shell), and Exercises (exercise details and starter templates). The REPL supports colon-prefixed commands just like the TUI (:compile, :verify, :help, etc.) and can also accept AICL code directly.

2.6 Status Bar

The status bar at the very bottom shows: current filename, cursor position (line:column), selected target language, and AICL version number.

3. Editor Features

The AICL Editor provides a complete set of editing features comparable to Python's IDLE, adapted for the AICL language:

Feature	Description	Shortcut
Syntax Highlighting	Keywords (cyan), Types (green), Comments (gray)	Automatic
Auto-indentation	4 spaces after colons, extra indent for sub-sections	Automatic
Line Numbers	Displayed on the left side of the editor	Always visible
Tab/Shift+Tab	Indent or outdent selected lines	Tab / Shift+Tab
Find and Replace	Search within the current file	Ctrl+F
Multiple File Tabs	Open and edit multiple files simultaneously	Click tabs
New File	Create a new untitled .aicl file	New button
Open File	Load a .aicl file from your computer	Open button
Save/Download	Download the current file as .aicl	Save button

Feature	Description	Shortcut
Auto-complete	Suggests AICL keywords as you type	Tab
Copy Generated Code	Copy compiled source or tests to clipboard	Copy button

4. Compilation and Verification

4.1 Compiling

Click the **Compile** button or use the REPL command `:compile` to compile the current AICL source. Select the target language from the dropdown (Python, Rust, JavaScript, Go) before compiling. The output panel shows the compilation result including audit coverage, TODO count, and proof validity. The Code tab displays the generated source and test files.

4.2 Verifying

Click **Verify** to check specification quality before compiling. The verifier runs three levels of checks: Completeness (all required elements present), Coherence (no contradictions), and Satisfaction (implementable and testable). Each check shows PASS, WARN, or FAIL status.

4.3 Auditing

Click **Audit** after compiling to see the audit report. The report shows total artifacts, artifacts with provenance, orphan count, and coverage percentage. The target is always 100% coverage with zero orphans.

4.4 Provenance Explanation

Click **Explain** to see why each line of generated code was produced. The explanation shows the provenance decision type, the source specification element, and the confidence level. All decisions are deterministic and fully auditable.

5. Examples

Four built-in examples demonstrate progressively more advanced AICL features:

Example	Levels	Description	Artifacts
Blue Square	1	Simple graphics application	35 artifacts, 100% audit
Pong Game	1-6	Game with entities, behaviors, conditions, events, parallelism	52 artifacts, 100% audit

Example	Levels	Description	Artifacts
Chat App	1-9	Full chat with security, optimization, learning	58 artifacts, 100% audit
Chess Game	1-9	Complex state management with network	59 artifacts, 100% audit

6. Exercises

Five progressive exercises teach AICL from basics to advanced. Each loads a starter template with TODO markers that you complete:

#	Title	Description	What You Learn
1	Hello World	Create a minimal AICL program	Goal, Layer, Validation
2	Risk and Recovery	Add error handling to a file reader	Risk/Recovery pairs
3	Entities and Behaviors	Define data structures with actions	Entity, Behavior, Action
4	Conditions and Events	Add rules and event handlers	When/Then, On/Action
5	Full Application	Build a complete chat application	All 9 levels combined

To start an exercise, click the Exercises tab in the left panel, then click the exercise title. The starter template loads in the editor with TODO markers showing where to add code. Complete the TODOs and click Compile to test your solution. Click Verify to check specification quality.

7. Interactive REPL

The REPL (Read-Eval-Print Loop) at the bottom of the editor provides an interactive command interface. Type colon-prefixed commands for actions, or type AICL code directly to add it to the current file. The REPL supports the same commands as the CLI TUI:

```
:help Show all REPL commands
:compile Compile current file
:verify Verify specification
:audit Audit compilation
:explain Explain provenance
:tree Architecture tree
:optimize Optimize architecture
:clear Clear output
:load Load an example
:exercises List exercises
```

8. Keyboard Shortcuts

Shortcut	Action
Ctrl+F	Find and Replace
Tab	Indent / Auto-complete
Shift+Tab	Outdent
Ctrl+S	Save/Download file

9. Troubleshooting

Compilation errors - Use Verify before Compile to catch specification issues early.

Empty output - Make sure you clicked Compile first. Audit and Explain require a compilation result.

File not loading - Ensure the file has a .aicl extension and valid UTF-8 encoding.

REPL not responding - Click inside the REPL input field to focus it.

Copy button not working - Your browser may block clipboard access. Use Ctrl+C manually.